

Q1. Smart Document Editor using String, StringBuilder and StringBuffer**CODE**

```
import java.util.Scanner;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String doc=sc.nextLine(),target=sc.nextLine(),rep=sc.nextLine();
        int chars=doc.length();
        int words=doc.trim().isEmpty()?0:doc.trim().split("\\s+").length;
        int sentences=doc.trim().isEmpty()?0:doc.split("[.!?]+").length;
        StringBuffer history=new StringBuffer();
        history.append("Loaded document.\n");
        String replaced=doc.replaceAll(target,rep);
        history.append("Replaced").append(target).append(" 'with'").append(rep).append("''.\n");
        StringBuilder finalDoc=new StringBuilder();
        for(String s:replaced.split("(?<=[.!?])|(?=[.!?])")){
            finalDoc.append(new StringBuilder(s).reverse());
        }
        history.append("Reversed sentence.");
        System.out.println("Original: "+doc);
        System.out.println("Chars: "+chars+"|Words: "+words+"|Sentences: "+sentences);
        System.out.println("\nHistory:\n"+history);
        System.out.println("\nFinal:\n"+finalDoc);
    }
}
```

INPUT

```
Hello world!Welcome to Java programming.Java is awesome!
Java
Python
```

OUTPUT

```
(no output)
```

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

class Main{
    public static void main(String[] args){
        List<Patient>list=new ArrayList<>();
        list.add(new Patient(101,"Alice","Cardiology"));
        list.add(new Patient(102,"Sam","Neurology"));
        list.add(new Patient(103,"Jaison","Cardiology"));
        System.out.println("All Patients:");
        list.forEach(System.out::println);
        for(Patient p:list){
            if(p.id==102)p.name="robert";
        }
        list.removeIf(p->p.id==103);
        System.out.println("\nCardiology Patients:");
        for(Patient p:list){
            if(p.department.equalsIgnoreCase("Cardiology"))System.out.println(p);
        }
        if(!list.isEmpty()){
            System.out.println("\nFirst:"+list.get(0));
            System.out.println("Last:"+list.get(list.size()-1));
        }
    }
}

class Patient{
    int id;
    String name,department;
    Patient(int id,String name,String department){
        this.id=id;
        this.name=name;
        this.department=department;
    }
    public String toString(){
        return id+ " "+name+" "+department;
    }
}
```

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Queue<String>queue=new LinkedList<>(Arrays.asList("Alice","Sam","jaison"));
        System.out.println("Regular Queue:"+queue);
        System.out.println("Next:"+queue.peek());
        System.out.println("Boarded:"+queue.poll());
        System.out.println("Remaining:"+queue);
        PriorityQueue<Passenger>vipQueue=new PriorityQueue<>();
        vipQueue.add(new Passenger("sam",2));
        vipQueue.add(new Passenger("jaison",1));
        System.out.println("\nPriority Queue:");
        while(!vipQueue.isEmpty()){
            System.out.println("Next:"+vipQueue.peek());
            System.out.println("Boarded:"+vipQueue.poll());
            System.out.println("Remaining count:"+vipQueue.size());
        }
    }
}
class Passenger implements Comparable<Passenger>{
    String name;
    int priority;
    Passenger(String name,int Priority){
        this.name=name;
        this.priority=priority;
    }
    public int compareTo(Passenger p){
        return Integer.compare(this.priority,p.priority);
    }
    public String toString(){
        return name+(priority==1?"(VIP)":"" );
    }
}
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Set<String>insertionOrderSet=new LinkedHashSet<>();
        insertionOrderSet.add("CERT101");
        insertionOrderSet.add("CERT102");
        insertionOrderSet.add("CERT103");
        insertionOrderSet.add("CERT101");
        System.out.println("Certificates(Insertion Order):"+insertionOrderSet);
        Set<String>sortedSet=new TreeSet<>(insertionOrderSet);
        System.out.println("Certificates(Sorted Order):"+sortedSet);
        System.out.println("Total Unique Certificate:"+insertionOrderSet.size());
        String checkId="CERT102";
        System.out.println("Is"+checkId+"valid?" +insertionOrderSet.contains(checkId));
        String revokedId="CERT103";
        insertionOrderSet.remove(revokedId);
        System.out.println("After Revoking"+revokedId+" "+insertionOrderSet);
    }
}
```

OUTPUT

(no output)

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;
class Main{
    public static void main(String[] args){
        Map<Integer,Product>inv=new HashMap<>();
        inv.put(101,new Product("Laptop","Electronics",800.0,10));
        inv.put(102,new Product("Phone","Electronics",500.0,15));
        inv.put(103,new Product("Desk","Furniture",150.0,5));
        if(inv.containsKey(101))inv.get(101).qty=12;
        System.out.println("Search 102:"+inv.get(102));
        double total=0;
        for(Product p:inv.values())total+=p.price*p.qty;
        System.out.println("Total Value:"+total);
        System.out.println("\nElectronics:");
        for(Product p:inv.values()){
            if(p.cat.equalsIgnoreCase("Electronics"))System.out.println(p);
        }
        inv.remove(103);
        System.out.println("\nAll Products:");
        for(Map.Entry<Integer,Product>e:inv.entrySet()){
            System.out.println("ID:"+e.getKey()+"|"+e.getValue());
        }
    }
}
class Product{
    String name,cat;
    double price;
    int qty;
    Product(String n,String c,double p,int q){
        name=n;cat=c;price=p;qty=q;
    }
    public String toString(){
        return name+"("+cat+")-"+price+"|Qty:"+qty;
    }
}
```

OUTPUT

(no output)